

# Lezione 09: Ereditarietà e Pattern Singleton

Questo notebook raccoglie tutto il codice della Lezione 09:

- SimpleDate/src/Language.java
- SimpleDate/src/Date.java
- SimpleDate/src/Birthday.java
- SimpleDate/src/MainDate.java
- SimpleDate/src/TestDate.java

Struttura dei file della lezione (path dalla cartella radice):

```
Programmazione-II/  
└─ codice-commentato/  
    └─ Lezione09/  
        └─ SimpleDate  
            └─ src  
                ├── Birthday.java  
                ├── Date.java  
                ├── Language.java  
                ├── MainDate.java  
                └─ TestDate.java
```

Argomenti trattati:

- Introduzione all'ereditarietà in Java tramite la parola chiave `extends`
- Derivazione della sottoclasse `Birthday` dalla superclasse `Date`
- Chiamata esplicita al costruttore della superclasse mediante `super(...)`
- Implementazione del pattern creazionale Singleton con costruttore privato ( `private Birthday()` ) e istanza condivisa restituita da `getInstance()`
- Confronto tra riferimenti d'istanza singleton ( `bDay == day1` )

## 1. Enum `Language`

```
In [1]: enum Language { IT, US }
```

## 2. Classe `Date`

```
In [2]: class Date {  
    private int day;  
    private int month;  
    private int year;  
    static final String FORMAT_IT = "dd/mm/yyyy";  
    static final String FORMAT_US = "mm/dd/yyyy";  
    private Language lang;  
    private static final String[] MONTHS_IT = { "gennaio", "febbraio", "marzo", "aprile", "maggio",  
    "giugno", "luglio", "agosto", "settembre", "ottobre", "novembre", "dicembre" };  
    private static final String[] MONTHS_US = { "January", "February", "March", "April", "May", "June",  
    "July", "August", "September", "October", "November", "December" };  
  
    public Date(int day, int month, int year) {  
        this.day = day;  
        this.month = month;  
        this.year = year;  
        verify();  
        lang = Language.IT;  
    }  
    public Date(int day, int month) {  
        this(day, month, 2025);  
    }  
    public Date(Date other) {  
        this.day = other.day;
```

```

        this.month = other.month;
        this.year = other.year;
        verify();
        lang = other.lang;
    }

    void verify() {
        if (year < 0 || month < 1 || month > 12)
            System.out.println("Illegal date!"); // Illegal date!
        else
            if (day < 1 || day > daysPerMonth(month))
                System.out.println("Illegal date!"); // Illegal date!
    }

    public int getDay() { return day; }
    public int getMonth() { return day; } // Nota didattica: getMonth() restituisce day in questa
versione
    public int getYear() { return day; } // Nota didattica: getYear() restituisce day in questa versione
    public Language getlang() { return lang; }

    public void setDay(int day) {
        this.day = day;
        verify();
    }
    public void setMonth(int month) {
        this.month = month;
        verify();
    }
    public void setYear(int year) {
        this.year = year;
        verify();
    }
    public void setAmerican() { lang = Language.US; }

    String printFormat() {
        return lang == Language.IT ? FORMAT_IT : FORMAT_US;
    }

    String getMonthAsString() {
        return (lang == Language.IT ? MONTHS_IT : MONTHS_US)[month-1];
    }

    String prettyPrint() {
        return lang == Language.IT ?
            day + " " + getMonthAsString() + " " + year :
            getMonthAsString() + " " + day + ", " + year;
    }

    String print() {
        return (lang == Language.IT ? day + "/" + month : month + "/" + day) + "/" + year;
    }

    public String toString() {
        return print();
    }

    static int daysPerMonth(int month) {
        int days;
        switch(month) {
            case 4:
            case 6:
            case 9:
            case 11:
                days = 30;
                break;
            case 2:
                days = 28;
                break;
            default:
                days = 31;
                break;
        }
        return days;
    }

```

```
}  
}
```

### 3. Classe `BirthDay` (Sottoclasse Singleton)

```
In [3]: class BirthDay extends Date {  
        private static BirthDay instance = null;  
  
        private BirthDay() {  
            super(1, 1, 1970);  
        }  
  
        public static BirthDay getInstance() {  
            if (instance == null)  
                instance = new BirthDay();  
            return instance;  
        }  
    }
```

### 4. Classe `TestDate` ed Esecuzione

```
In [4]: class TestDate {  
        private static Date date = new Date(1, 1, 1970);  
  
        private static void printFormatTest() {  
            date.setAmerican();  
            assert date.printFormat().equals(Date.FORMAT_US);  
            System.out.println("printFormatTest: OK"); // printFormatTest: OK  
        }  
  
        public static void main(String[] args) {  
            printFormatTest();  
        }  
    }  
TestDate.main(null);
```

printFormatTest: OK

### 5. Classe `MainDate` ed Esecuzione

```
In [5]: class MainDate {  
        public static void main(String[] args) {  
            Date d1 = new Date(7, 1, 2025);  
            Date d2 = new Date(20, 2);  
            Date d3 = new Date(d2);  
            d2.setAmerican(); // set format to US  
            System.out.println(d1.toString() + " " + d1.printFormat()); // 7/1/2025 dd/mm/yyyy  
            System.out.println(d2.toString() + " " + d2.printFormat()); // 2/20/2025 mm/dd/yyyy  
            System.out.println(d3.toString() + " " + d3.printFormat()); // 20/2/2025 dd/mm/yyyy  
            System.out.println(d1.prettyPrint()); // 7 gennaio 2025  
            System.out.println(d2.prettyPrint()); // February 20, 2025  
            System.out.println(d3.prettyPrint()); // 20 febbraio 2025  
            //  
            Date[] dates = {d1, d2, d3, new Date(14, 2, 2024)};  
            for (int i = dates.length-1; i >= 0; i--)  
                System.out.println(dates[i].toString() + ": " + dates[i].getMonthAsString()); // 14/2/2024:  
            febbraio \n 20/2/2025: febbraio \n 2/20/2025: February \n 7/1/2025: gennaio  
  
            for (Date date : dates)  
                System.out.println(date.toString() + ": " + date.getMonthAsString()); // 7/1/2025: gennaio \n  
            2/20/2025: February \n 20/2/2025: febbraio \n 14/2/2024: febbraio  
  
            //  
            BirthDay bDay = BirthDay.getInstance();  
            // it.oop.core.BirthDay day1 = new it.oop.core.BirthDay(); // compile-time error  
            BirthDay day1 = BirthDay.getInstance();  
            System.out.println("bDay != day1: " + (bDay != day1)); // bDay != day1: true  
            Date today = new Date(27, 10, 202);  
            // today.getMonth() restituisce 27 e today.getYear() restituisce 27 a causa del getter:  
            Date tomorrow = new Date(today.getDay()+1, today.getMonth(), today.getYear()); // verify()
```

```
stampa: Illegal date!
    System.out.println("today: " + today.toString()); // today: 27/10/202
    System.out.println("tomorrow: " + tomorrow.toString()); // tomorrow: 28/27/27
}
MainDate.main(null);
```

```
7/1/2025 dd/mm/yyyy
2/20/2025 mm/dd/yyyy
20/2/2025 dd/mm/yyyy
7 gennaio 2025
February 20, 2025
20 febbraio 2025
14/2/2024: febbraio
20/2/2025: febbraio
2/20/2025: February
7/1/2025: gennaio
7/1/2025: gennaio
2/20/2025: February
20/2/2025: febbraio
14/2/2024: febbraio
bDay ?= day1: true
Illegal date!
today: 27/10/202
tomorrow: 28/27/27
```